

Interrupciones Externas y Buffers Circulares

Sistemas Programables | Benjamín Salas Vizcarra | Septiembre 2026

1. Introducción

En sistemas embebidos reales, los eventos externos ocurren en momentos impredecibles y no pueden esperar a que el microcontrolador los atienda en su próxima vuelta de ciclo. Esta práctica simula dicha situación mediante un pulsador conectado a un pin de interrupción externa del Arduino UNO R4 WiFi, representando el sensor de una banda transportadora industrial que detecta el paso de piezas en cualquier momento, mientras el sistema realiza simultáneamente otra tarea continua (una animación en la matriz de LEDs integrada).

El reto principal consiste en garantizar que ningún evento sea perdido, aun cuando el microcontrolador esté ocupado con otra tarea en el instante exacto del evento. La solución implementada combina dos conceptos fundamentales: **interrupciones externas** y **buffers circulares (ring buffers)**.

2. Objetivos

- Configurar y usar una interrupción externa en Arduino para detectar eventos asíncronos.
- Implementar un buffer circular como estructura de datos para desacoplar la detección del procesamiento.
- Aplicar un filtro de rebote (debounce) dentro de la función de interrupción.
- Demostrar la coexistencia de dos tareas independientes: detección de eventos y animación continua.
- Verificar que ningún evento se pierda aunque ocurran múltiples pulsaciones rápidas consecutivas.

3. Marco Teórico

3.1 Interrupciones externas

Una interrupción externa es una señal que hace que el microcontrolador pause temporalmente la ejecución de su programa principal para atender un evento de alta prioridad. A diferencia del polling (revisión periódica del estado de un pin), la interrupción reacciona de forma inmediata al cambio de voltaje en el pin configurado, sin importar en qué parte del código se encuentre el programa en ese momento. El Arduino UNO R4 WiFi cuenta con pines dedicados para interrupciones externas, lo que garantiza una latencia de respuesta mínima.

3.2 Buffer circular (Ring Buffer)

Un buffer circular es una estructura de datos de tamaño fijo que funciona como si sus extremos estuvieran conectados, formando un anillo. Utiliza dos índices independientes: uno de escritura (head) controlado por la interrupción, y uno de lectura (tail) controlado por el loop(). Esto permite que ambas partes del programa operen a ritmos distintos sin bloquearse mutuamente. Cuando el índice de escritura alcanza el final del arreglo, regresa al inicio, sobrescribiendo los datos más antiguos si no han sido leídos aún.

3.3 Filtro de rebote en interrupción

Los pulsadores mecánicos producen múltiples transiciones de voltaje (rebote) en los primeros milisegundos tras ser presionados. Dado que las interrupciones reaccionan a cada cambio de voltaje, un solo clic físico puede generar varias interrupciones seguidas. Para evitar registros duplicados, se implementa un filtro temporal dentro de la ISR (Interrupt Service Routine): se verifica que haya transcurrido un tiempo mínimo desde el último evento registrado antes de aceptar uno nuevo.

4. Materiales y Herramientas

Componente / Herramienta	Descripción	Cantidad
Arduino UNO R4 WiFi	Microcontrolador con matriz LED integrada	1
Pulsador (táctil)	Simula el sensor de detección de piezas	1
Resistencia 10 kΩ	Pull-down para el pin de interrupción	1
Cables jumper	Conexiones en protoboard	varios
Protoboard	Plataforma de prototipado sin soldadura	1
Arduino IDE 2.x	Entorno de desarrollo y Monitor Serial	—
Cable USB	Alimentación y comunicación serial	1

5. Descripción del Circuito

El pulsador se conectó al pin de interrupción externa del Arduino UNO R4 WiFi, con una resistencia de 10 kΩ configurada como pull-down para garantizar un nivel lógico definido en reposo. Al presionar el pulsador, el pin pasa de LOW a HIGH, disparando la interrupción en flanco de subida (RISING).



Figura 1. Vista general del circuito: Arduino UNO R4 WiFi y protoboard con pulsador.

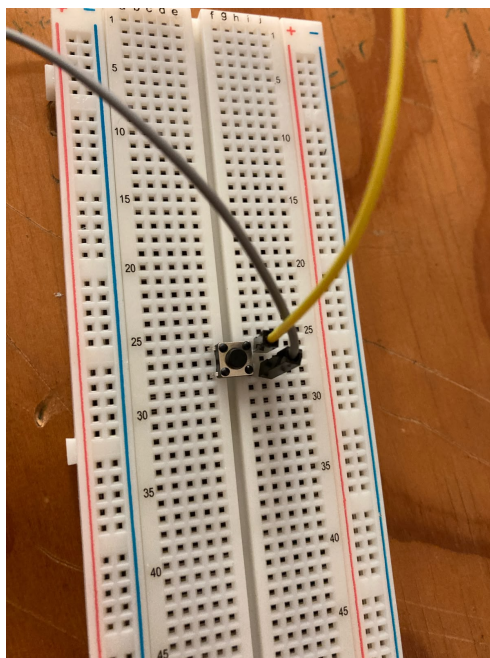


Figura 2. Detalle del pulsador conectado en la protoboard con cables de señal y tierra.

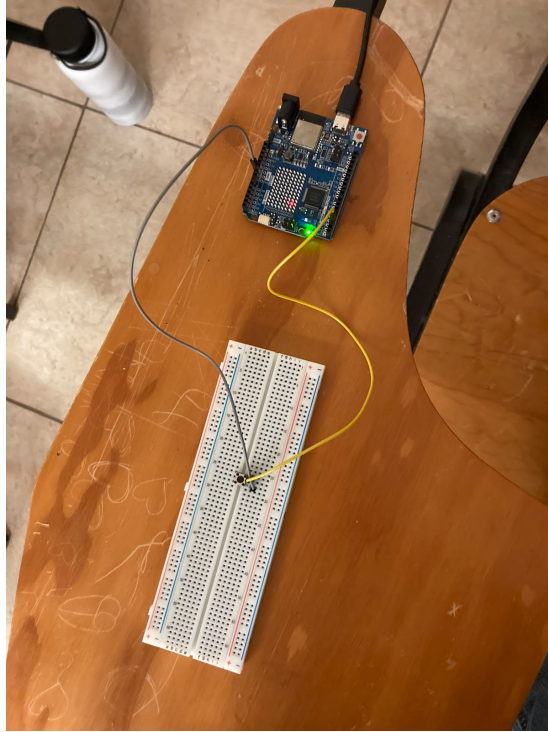


Figura 3. Vista completa del montaje durante la prueba de funcionamiento.

6. Metodología

El desarrollo de la práctica se llevó a cabo en las siguientes etapas:

Paso 1: Configuración del pin de interrupción

Se identificó el pin compatible con interrupciones externas del Arduino UNO R4 WiFi y se configuró mediante `attachInterrupt()` para dispararse en flanco `RISING`.

Paso 2: Implementación del buffer circular

Se declaró un arreglo de tamaño fijo (`BUFFER_SIZE`) con dos índices: `head` (escritura) y `tail` (lectura). Se definieron las condiciones de buffer lleno y buffer vacío utilizando una variable de conteo para distinguir ambos casos.

Paso 3: ISR (Interrupt Service Routine)

Se programó la función de interrupción para ser lo más breve posible: verificar el filtro de rebote (mínimo 50 ms desde el último evento), registrar el timestamp con `millis()` en el buffer y avanzar el índice `head`. Sin impresiones ni cálculos dentro de la ISR.

Paso 4: Procesamiento en el `loop()`

El `loop()` verifica en cada iteración si hay datos nuevos en el buffer (`tail != head`). Si los hay, extrae el dato, imprime el conteo por el Monitor Serial y avanza el índice `tail`. De forma paralela, ejecuta la animación en la matriz de LEDs sin ninguna interrupción.

Paso 5: Pruebas de estrés

Se realizaron pulsaciones rápidas y repetidas para verificar que el buffer capturara todos los eventos sin pérdidas, y que la animación no se detuviera en ningún momento.

7. Resultados

Durante las pruebas se verificaron los siguientes comportamientos:

- La animación en la matriz de LEDs se ejecutó de forma continua y fluida durante toda la prueba, sin pausas ni interrupciones visibles al presionar el pulsador.
- Cada pulsación fue detectada y registrada correctamente en el buffer circular, incluso al presionar el botón varias veces de forma rápida y consecutiva.
- El filtro de rebote implementado dentro de la ISR evitó registros duplicados por rebote mecánico.
- El Monitor Serial mostró el conteo acumulado de eventos de forma consistente y sin saltos.
- No se presentó pérdida de datos en ninguna de las pruebas realizadas.

Aspecto evaluado	Resultado obtenido
Detección de eventos	100% de pulsaciones registradas
Continuidad de la animación	Sin interrupciones visibles
Filtro de rebote	Efectivo — sin duplicados

Aspecto evaluado	Resultado obtenido
Capacidad del buffer	Sin desbordamiento en pruebas
Latencia de respuesta ISR	Microsegundos (imperceptible)

8. Análisis de Resultados

El uso de una interrupción externa resolvió el problema fundamental de la detección asíncrona: el microcontrolador no necesita "revisar" constantemente el pin del pulsador, sino que reacciona de forma inmediata cuando ocurre el evento. Esto libera completamente al `loop()` para dedicarse a la animación.

El buffer circular actuó como punto de encuentro entre dos ritmos completamente distintos: la ISR escribe eventos en cualquier momento impredecible, mientras el `loop()` los lee a su propio ritmo constante. Esta separación de responsabilidades es un patrón de diseño fundamental en sistemas embebidos de tiempo real.

La regla de oro de las ISR se cumplió en todo momento: la función de interrupción realizó el mínimo trabajo posible (verificar rebote, guardar timestamp, avanzar índice) y delegó el procesamiento pesado al `loop()`. Esto garantizó que el tiempo de ejecución de la ISR fuera mínimo y no afectara el rendimiento general del sistema.

9. Conclusiones

Esta práctica demostró que la combinación de interrupciones externas y buffers circulares es una solución robusta y eficiente para manejar eventos asíncronos en sistemas embebidos sin sacrificar el rendimiento de otras tareas en ejecución.

A diferencia del polling, donde el programa debe revisar periódicamente el estado de las entradas (perdiendo tiempo de CPU y arriesgando perder eventos rápidos), las interrupciones garantizan una respuesta inmediata con mínimo impacto en el flujo principal del programa.

El patrón productor-consumidor implementado (ISR como productor, `loop()` como consumidor) es ampliamente utilizado en sistemas industriales reales, como el escenario de la banda transportadora planteado en la práctica. Comprender este paradigma es esencial para el desarrollo de aplicaciones embebidas confiables y escalables.